

0 OBJETIVOS

- Diseñar soluciones computacionales para problemas.
- Estimar costos de las soluciones planteadas.
- Implementar soluciones.

Se premiarán las mejores soluciones y se castigarán las peores, en cuanto a optimización y eficiencia en tiempo y espacio.

1 CONDICIONES GENERALES

El proyecto se divide en tres partes independientes entre sí. Este documento describe la PARTE I. Cada parte contiene un problema a resolver mediante soluciones implementadas en *Java* o *Python*.

Para cada problema se pide:

- Descripción de la solución.
- Análisis temporal y espacial.
- Una implementación en Java o Python

2 DESCRIPCIÓN DEL PROBLEMA

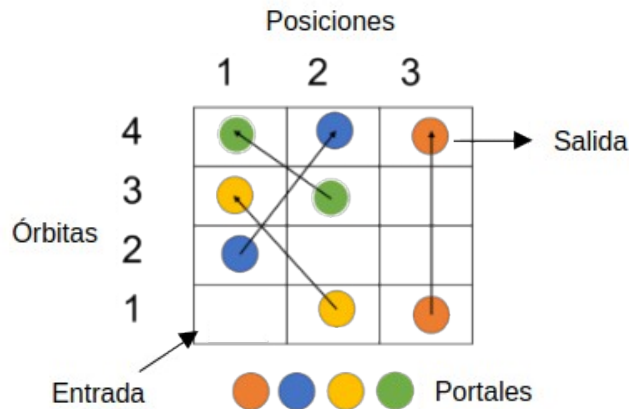
Un investigador en física del planeta Dirac33 descubrió una nueva partícula subatómica llamada Termion. Para investigar los patrones de movimiento de esta nueva partícula, diseñó un campo rectangular compuesto por n órbitas, cada una de las cuales cuenta con m posiciones. Un termion puede moverse dentro de una órbita desde la posición en la que se encuentra hacia una posición adyacente, consumiendo una cantidad de energía que es fija dentro de cada órbita. Algunas posiciones del campo también incluyen portales de teletransportación que le permiten a un termion viajar de una órbita específica hacia otra (en una sola dirección) sin consumir energía. El objetivo del investigador es analizar si un termion puede encontrar la ruta de menor consumo de energía entre la entrada, ubicada en la primera posición de la primera órbita, y la salida del campo, ubicada en la última posición de la última órbita.

Problema:

Dado un diseño de campo, determinar si es posible para un termion llegar del comienzo al final del campo y de ser así determinar la energía mínima para llegar al final

Ejemplo:

Dado el campo con 4 órbitas determinado por la siguiente figura y dado que las energías para moverse dentro de cada órbita son $e_1=2$, $e_2=1$, $e_3=3$ y $e_4=0$:



Dos rutas directas de la entrada a la salida del campo están dadas por los siguientes movimientos (se marcan en rojo los destinos de los movimientos de teletransportación):

Ruta 1: (1,1)-(1,2)-(3,1)-(3,2)-(4,1)-(4,2)-(4,3)

Energía Ruta 1: 5

Ruta 2: (1,1)-(1,2)-(1,3)-(4,3)

Energía Ruta 2: 4

La ruta 2 es la que le permite al termion llegar al destino con el menor consumo de energía.

3 ENTRADA Y SALIDA DE DATOS

En todas las soluciones que se presenten, la lectura de los datos de entrada se hace por la entrada estándar; así mismo, la escritura de los resultados se hace por la salida estándar.

Puede suponer que ninguna línea de entrada tiene espacios al principio o al final, y que los datos que se listan en cada línea están separados por exactamente un espacio.

A continuación, se establecen parámetros que definen su tamaño y formato de lectura de los datos, tanto de entrada como de salida.

Descripción de la entrada

La primera línea de entrada especifica el número de casos de prueba que contiene el archivo. El programa debe terminar su ejecución, una vez termine de resolver la cantidad de casos de prueba dados por este número.

Cada caso contiene varias líneas. La primera línea de cada caso contiene 3 enteros, n es el número de órbitas, m es el número de fases por cada órbita y p es el número total de portales entre órbitas.

La segunda línea contiene una lista de n enteros que corresponden a los valores de consumo de energía e_i . Las siguientes líneas contienen la descripción de los portales. Cada portal es descrito por 4 enteros x_s, y_s, x_e, y_e .

Límites de tamaños de entrada para pruebas:

- $1 \leq n \leq 10^3$, $1 \leq m \leq 10^3$, $0 \leq p \leq nm$
- Para cada órbita i : $0 \leq e_i \leq 10^3$
- Para cada portal: $1 \leq x_s < x_e \leq n$, $1 \leq y_s \leq m$, $1 \leq y_e \leq m$

Descripción de la salida

Para cada caso de prueba, imprimir:

1. NO EXISTE, si no existe ninguna ruta.
2. El costo de la ruta óptima (i.e. menor energía gastada) para llegar al destino.

Ejemplo de entrada / salida

Entrada	Salida
3 4 3 4 2 1 3 0 1 2 3 1 1 3 4 3 2 1 4 2 3 2 4 1 5 3 3 5 17 8 1 4 1 3 3 3 3 1 5 2 3 2 5 1 6 3 3 5 17 8 1 4 2 1 3 3 3 3 1 5 2 3 2 5 1	4 26 NO EXISTE

Nota: Se van a diseñar casos de prueba para valores de n mucho más grandes y dentro de los valores establecidos en el enunciado. Los casos mostrados en este documento son demostrativos de la estructura de entrada/salida esperada.

4 ENTREGABLES

El proyecto puede desarrollarse por grupos de hasta tres estudiantes de la misma sección. La entrega se hace por bloque neon (una sola entrega por grupo de trabajo).

El grupo debe entregar, por bloque neon, un archivo de nombre `proyectoDalgoP1.zip`. Este archivo es una carpeta de nombre `proyectoDalgoP1`, comprimida en formato `.zip`, dentro de la cual hay archivos fuente de soluciones propuestas y archivos que documentan cada una de las soluciones.

4.1 Archivos fuente de soluciones propuestas

Todos los programas implementados en *Java* o en *Python*

Para el problema:

- Entregar un archivo de código fuente en *Java* (`.java`) o *python* (`.py`) con su código fuente de la solución que se presenta.
- Incluir como encabezado de cada archivo fuente un comentario que identifique el (los) autor(es) de la solución.
- Denominar `ProblemaP1.java` o `ProblemaP1.py` el archivo de la solución que se presente.

Nótese que, si bien puede utilizarse un *IDE* como *Eclipse* o *Spyder* durante el desarrollo del proyecto, la entrega requiere incluir solo un archivo por cada solución. El archivo debe poderse

compilar y ejecutar independientemente (sin depender de ninguna estructura de directorios, librerías no estándar, etc.).

4.2 Archivos que documentan la solución propuesta

La solución al problema debe acompañarse de un archivo de máximo 3 páginas que la documente, con extensión .pdf. El nombre del archivo debe ser el mismo del código correspondiente (ProblemaP1.pdf).

Un archivo de documentación debe contener los siguientes elementos:

- 0 *Identificación*
Nombre de autor(es)
- 1 *Algoritmo de solución*
Explicación del algoritmo elegido. Generar al menos una gráfica que apoye la explicación del algoritmo implementado. No se debe copiar y pegar código fuente como parte de la explicación del algoritmo.
- 2 *Análisis de complejidades espacial y temporal*
Cálculo de complejidades y explicación de estas.

La nota del informe corresponde a un 50% de la nota total de la entrega del proyecto, solamente si se entrega el código fuente de la solución implementada. En caso de no entregar el código fuente, no se hará evaluación del informe y la nota del proyecto será cero.

Además de la pertinencia del texto como explicación de la solución implementada, se evaluará la calidad en la redacción del texto y en el diseño de las gráficas. Se evaluará también que la explicación del algoritmo integre los conceptos relacionados con las técnicas de diseño de algoritmos cubiertas en el curso.

Téngase en cuenta que los análisis de 2 tienen sentido en la medida que la explicación de 1 sea clara y correcta.

4.3 Consideraciones sobre la entrega

Lea bien las recomendaciones de entrada/salida. Su proyecto será evaluado, en gran parte, por una máquina: si en la ejecución del programa el formato de salida no coincide perfectamente con lo requerido, la calificación de la ejecución será cero, sin importar la cantidad de código que haya desarrollado.

La forma en que se presenta la documentación debe respetarse, aunque su evaluación no sea tan automática como la del software. Hay que nombrar los archivos como se espera que se nombren, comprimirllos como se pide que se compriman, etc. Cualquier desviación en cuanto a lo que se pide produce deméritos en la calificación final.

El software se evalúa desde la línea de comandos de Linux. No hay problema en desarrollar en cualquier otro sistema (Mac, Windows, ...) siempre que se produzca código estándar en java o python. Cada problema se debe resolver en un (1) archivo.